COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:		**SECTION:**	
DATE PERFORMED:		**DATE SUBMITTED:**	
NAME:		**INSTRUCTOR:**	

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

ILO A

Screenshot

```cpp
#include <iostream>
#include <stack>

int main(){

    std::stack<int> stack1,stack2;
    std::cout << "Testing the stack STL" << std::endl;

    // isEmpty
    std::cout << "Is the stack empty? " << stack1.empty() << std::endl;

    // push
    stack1.push(10);
    std::cout << "The top of the stack is " << stack1.top() << std::endl;
    stack1.push(9);
    std::cout << "The top of the stack is " << stack1.top() << std::endl;
    stack1.push(8);
    std::cout << "The top of the stack is " << stack1.top() << std::endl;
    stack1.push(7);
    std::cout << "The top of the stack is " << stack1.top() << std::endl;

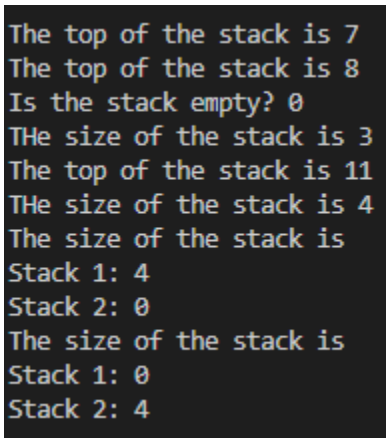
    // pop
    stack1.pop();
    std::cout << "The top of the stack is " << stack1.top() << std::endl;

    std::cout << "Is the stack empty? " << stack1.empty() << std::endl;
    std::cout << "THe size of the stack is " << stack1.size() << std::endl;

    stack1.emplace(11);
    std::cout << "The top of the stack is " << stack1.top() << std::endl;
    std::cout << "THe size of the stack is " << stack1.size() << std::endl;

    std::cout << "The size of the stack is " << std::endl;
    std::cout << "Stack 1: " << stack1.size() << std::endl;
    std::cout << "Stack 2: " << stack2.size() << std::endl;

    stack1.swap(stack2);
    std::cout << "The size of the stack is " << std::endl;
    std::cout << "Stack 1: " << stack1.size() << std::endl;
    std::cout << "Stack 2: " << stack2.size() << std::endl;
    return 0;
}
```

Output	
Analysis	By utilizing the C++ STL, it made it much easier to understand the stack operations because you can just call the functions and they will do the work for you. I can see that the functions called are doing their purpose like what is explained in the lecture. I think that LIFO principle is clearly shown by doing this ILO A.

ILO B1

Screenshot

```cpp
#include <iostream>

//global declarations
#define maxCap 100
int stackArr[maxCap];
int top = -1, newData;

//prototype functions
void push();
void pop();
void Top();
bool isEmpty();
bool isFull();
void display();

int main(){
    //main driver code
int choice;
    while(true){
        std::cout << "\n++++++++++++++++++++++++++++++++++++++++++++++++" << std::endl;
        std::cout << "Stack Operations \n";
        std::cout << "1. Push 2. Pop 3. Top 4. isEmpty 5. isFull 6. Display 7. End" << std::endl;
        std::cout << "++++++++++++++++++++++++++++++++++++++++++++++++\n";
        std::cin >> choice;

        switch (choice){
            case 1: push();
            break;
            case 2: pop();
            break;
            case 3: Top();
            break;
            case 4: std::cout << "is stack empty? " << isEmpty() << std::endl;
            break;
            case 5: std::cout << "Is the stack full? " << isFull() << std::endl;
            break;
            case 6: display();
            break;
            case 7: return 0;
            break;
            default: std::cout << "Invalid Choice" << std::endl;
        }
    }
}
```

```
    // ERROR CHECKING
    if(isFull()){
        std::cout << "Stack Overflow " << std::endl;
        return;
    }
    // PUSHING STACK
    std::cout << "\nEnter a new Value: " << std::endl;
    std::cin >> newData;
    // HOW TO INSERT THE DATA IN THE STACK
    stackArr[++top] = newData;
}

void pop(){
    // ERROR CHECKING
    if(isEmpty){
        std::cout << "Stack Underflow" << std::endl;
    }

    // DISPLAY THE VALUE THAT WE ARE GOING TO POP IN POP IN BLOOM
    std::cout << "POPPING: " << stackArr[top] << std::endl;

    // DECREMENT THE TOP VALUE FRPM THE STACK
    top--;
}

void Top(){
    // ERROR CATCHING
    if(isEmpty){
        std::cout << "The Stack is empty" << std::endl;
        return;
    }
    // CHECK TOP VALUE
    std::cout << "Top Element: " << stackArr[top] << std::endl;
}

void display() {
    if (isEmpty()) {
        std::cout << "Stack is empty." << std::endl;
        return;
    }

    std::cout << "Stack elements (TOP TO BOT):" << std::endl;
    for (int j = top; j >= 0; j--) {
        std::cout << stackArr[j] << " ";
    }
    std::cout << std::endl;
}
```

```cpp
//function definition
bool isEmpty(){
    if(top == -1) return true;
    return false;
}

bool isFull(){
    if (top == maxCap - 1) return true;
    return false;
}

void push(){
    if (isFull()){
        std::cout << "Stack Overflow!" << std::endl;
        return;
    }

    std::cout << "Enter a new value: " << std::endl;
    std::cin >> newData;
    stackArr[++top] = newData;
}

void pop(){
    if(isEmpty()){
        std::cout << "Stack Underflow!" << std::endl;
    }

    //display the value that we are going to pop.
    std::cout << "Popping: " << stackArr[top] << std::endl;
    //decrement the top value from the stack
    top--;
}

void Top(){
    //error catching:
    if(isEmpty()){
        std::cout << "The stack is empty." << std::endl;
        return;
    }

    std::cout << "Top Element: " << stackArr[top]<<std::endl;
}
```

```
void display(){
    if(isEmpty()){
        std::cout << "The stack is empty." << std::endl;
        return;
    }
    std::cout << "This is the content of your stack: ";

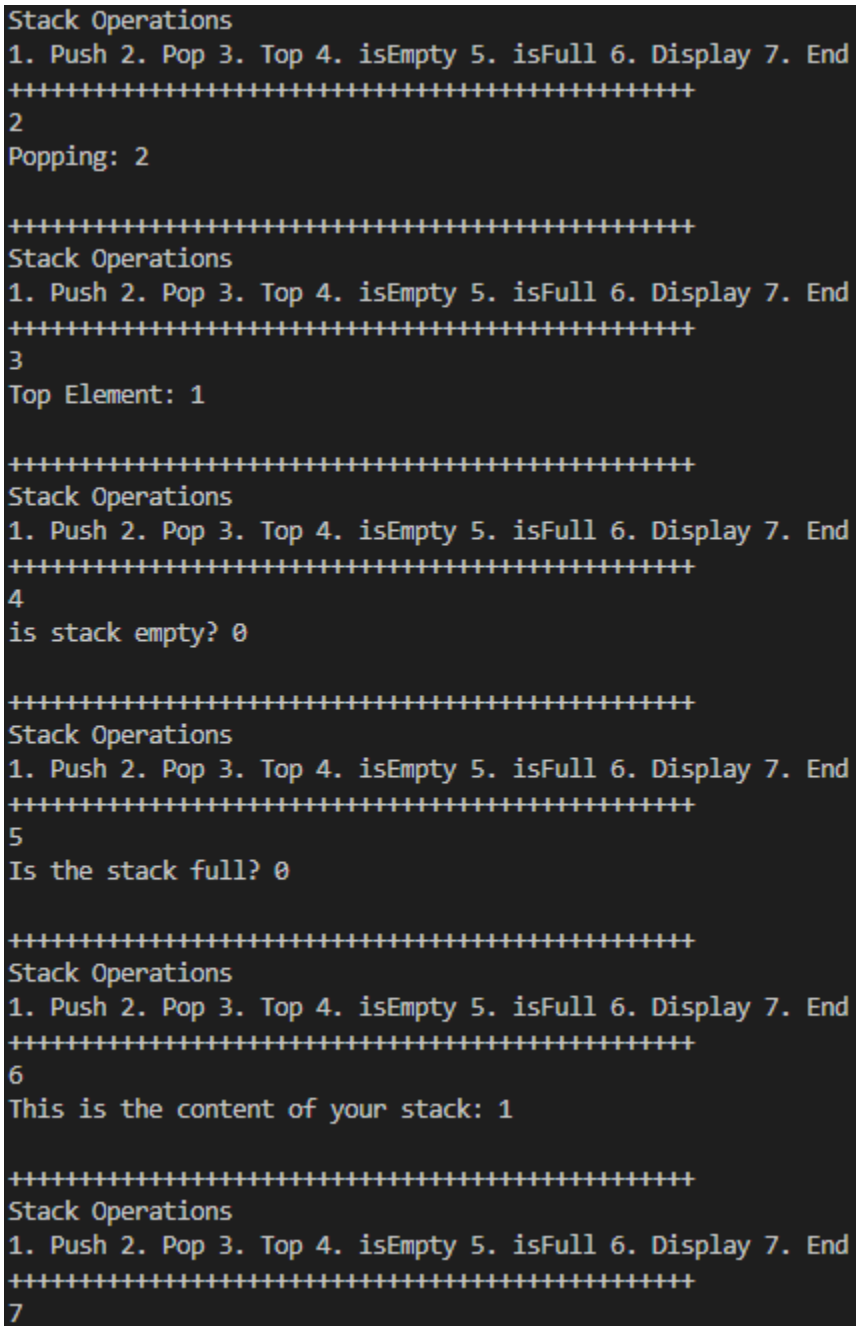
    for (int i = top; i >= 0; i--){
        std::cout << stackArr[i];
    }
    std::cout << std::endl;
}
```

Output

```
++++++++++++++++++++++++++++++++++++++++++++++++
Stack Operations
1. Push 2. Pop 3. Top 4. isEmpty 5. isFull 6. Display 7. End
++++++++++++++++++++++++++++++++++++++++++++++++
1
Enter a new value:
1

++++++++++++++++++++++++++++++++++++++++++++++++
Stack Operations
1. Push 2. Pop 3. Top 4. isEmpty 5. isFull 6. Display 7. End
++++++++++++++++++++++++++++++++++++++++++++++++
1
Enter a new value:

2
```

	
Analysis	Implementing an array stack helped me analyze indexes, most importantly the top index controls the movement of the elements especially when pushing and popping them.

ILO B2

main.cpp

Screenshot

```
#include <iostream>
#include "stack_ll.h"

int main() {

    std::cout << "Pushing Elements into the Stack\n";

    //pushing elements
    push('C');
    push('P');
    push('E');

    //Displaying the top
    Top<char>;
    displayALL<char>();
    std::cout << "Element Popping\n";

    //popping elements until empty
    while(!isEmpty<char>()) {
        std::cout<<"Popped: "<< pop<char>() <<std::endl;
    }

    //checks if the stack is underflowing
    std::cout << "\nUnderflow Condition\n";
    pop<char>();

    return 0;
}
```

stack.h

Screenshot

```h
#ifndef STACK_H
#define STACK_H

#include <iostream>

template <typename T>
class Node {
    public:
    T data; //stored value
    Node* next; //pointer to next
};

//pointer for the top
template <typename T>
Node<T>* head = nullptr;

//stack operations

//pushing
template <typename T>
void push(T newData) {
    //creating new node and assigning new data
    Node<T>* newNode = new Node<T>();
    newNode->data = newData;

    //points to the current top which will become the new top
    newNode->next = head<T>;
    head<T> = newNode;
}

//popping

template <typename T>
T pop() {
    //checks if stack is empty
    if (head<T> == nullptr) {
        std::cerr << "Error. Cannot pop from an empty stack." << std::endl;
    }

    //saving the top node
    Node<T>* temp = head<T>;
    T tempVal = temp->data;

    //move to the next node
    head<T> = head<T>->next;

    delete temp;
```

```
    return tempVal;
}

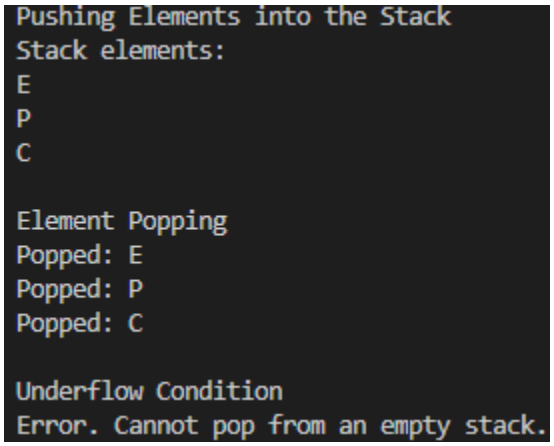
//isEmpty
template <typename T>
bool isEmpty() {
    return head<T> == nullptr;
}

//Top
template <typename T>
void Top() {
    if (head<T> == nullptr) {
        std::cout << "Stack is empty." << std::endl;
        return;
    }
    std::cout << "Top element: " << head<T>->data << std::endl;
}

//display all
template <typename T>
void displayALL() {
    if (head<T> == nullptr) {
        std::cout << "Stack is Empty.\n" << std::endl;
        return;
    }

    //traverse and display
    Node<T>* temp = head<T>;
    std::cout << "Stack elements: \n";
    while (temp != nullptr) {
        std::cout << temp->data << " \n";
        temp = temp->next;
    }
    std::cout << std::endl;
}

#endif
```

Output	
Analysis	Stacked Linked Lists are much more dynamic than arrays. I can continuously add elements without worrying about the size. Basically, new nodes become the top of the stack and the previous top node moves down. Making it possible for push and pop operations to work properly.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C

main.cpp

Screenshot

```cpp
#include <iostream>
#include <string>
#include <stack>
#include <unordered_map>
#include <iomanip>

#define MAX_SIZE 100

// matching pairs, assigning delimiters for character pairings
const std::unordered_map<char, char> pair = {
    {')', '('},
    {']', '['},
    {'}', '{'}
};

// categorizing the characters
// for opening characters
bool isOpening(char ch) {
    return ch == '(' || ch == '[' || ch == '{';
}

// for closing characters
bool isClosing(char ch) {
    return pair.count(ch) > 0;
}

// Stack using array
class Array {
private:
    //array that stacks pushed elements
    char arr[MAX_SIZE];
    int topIndex;

public:
    //top is always -1 of the array size
    Array() : topIndex(-1) {}

    //pushing function
    bool push(char val) {
        //checks if the top reaches max size, full returns false
        if (topIndex >= MAX_SIZE - 1) return false;
        arr[++topIndex] = val;
        return true;
    }
```

```
    //popping function
    char pop() {
        //if empty, returns null value
        if (isEmpty()) return '\0';
        return arr[topIndex--];
    }
    //top function
    char top() const {
        if (isEmpty()) return '\0';
        return arr[topIndex];
    }
    //isEmpty function
        //returns true if value is less than 0
    bool isEmpty() const {
        return topIndex < 0;
    }
};

bool checkArray(const std::string& expr) {
    Array stack;
    for (char ch : expr) {
        if (isOpening(ch)) {
            stack.push(ch);
        } else if (isClosing(ch)) {
            if (stack.isEmpty() || stack.pop() != pair.at(ch)) {
                return false;
            }
        }
    }
    return stack.isEmpty();
}

// Linked list
struct Node {
    char data;
    Node* next;
};

class LinkedList {
private:
    Node* topNode;

public:
    LinkedList() : topNode(nullptr) {}
```

```
    ~LinkedList() {
        while (!isEmpty()) {
            pop();
        }
    }
    //pushing
    void push(char val) {
        Node* newNode = new Node();
        newNode->data = val;
        newNode->next = topNode;
        topNode = newNode;
    }
    //popping
    char pop() {
        if (isEmpty()) return '\0';
        Node* temp = topNode;
        char poppedVal = temp->data;
        topNode = topNode->next;
        delete temp;
        return poppedVal;
    }
    //isEmpty
    bool isEmpty() const {
        return topNode == nullptr;
    }
};

bool checkLinkedList(const std::string& expr) {
    LinkedList stack;
    for (char ch : expr) {
        if (isOpening(ch)) {
            stack.push(ch);
        } else if (isClosing(ch)) {
            if (stack.isEmpty() || stack.pop() != pair.at(ch)) {
                return false;
            }
        }
    }
    return stack.isEmpty();
}
```

```cpp
//C++ STL
bool checkSTL(const std::string& expr) {
    std::stack<char> s;
    for (char ch : expr) {
        if (isOpening(ch)) {
            s.push(ch);
        } else if (isClosing(ch)) {
            if (s.empty() || s.top() != pair.at(ch)) {
                return false;
            }
            s.pop();
        }
    }
    return s.empty();
}

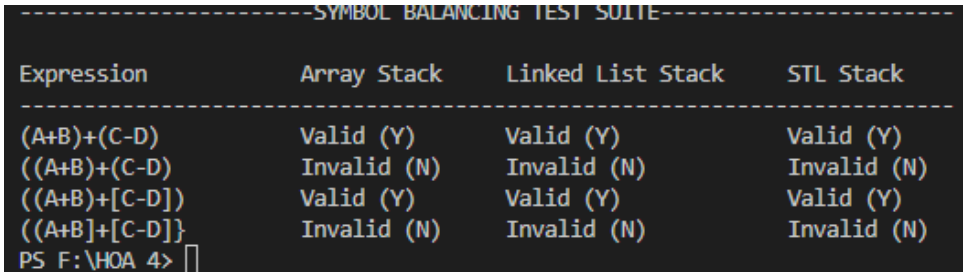
//MAIN FUNCTION
int main() {
    std::string testCases[] = {
        "(A+B)+(C-D)",
        "((A+B)+(C-D)",
        "((A+B)+[C-D])",
        "((A+B]+[C-D]}"
    };


    std::cout << "----------------------SYMBOL BALANCING TEST SUITE----------------------\n\n";

    std::cout << std::left
              << std::setw(22) << "Expression"
              << std::setw(16) << "Array Stack"
              << std::setw(22) << "Linked List Stack"
              << std::setw(16) << "STL Stack"
              << "\n";
    std::cout << "-------------------------------------------------------------------------\n";

    for (const std::string& expr : testCases) {
        bool resArray = checkArray(expr);
        bool resLL    = checkLinkedList(expr);
        bool resSTL   = checkSTL(expr);
        //formatting
        std::cout << std::left
                  << std::setw(22) << expr
                  << std::setw(16) << (resArray ? "Valid (Y)" : "Invalid (N)")
                  << std::setw(22) << (resLL    ? "Valid (Y)" : "Invalid (N)")
                  << std::setw(16) << (resSTL   ? "Valid (Y)" : "Invalid (N)")
                  << "\n";
    }
```

```cpp
    return 0;
}
```

Output	
Analysis	It was easy at first if you try to validate it by using three different cpp files with different formats. However, I thought of a more complicated way to make the activity which is by incorporating all three of them in one file. Basically, they all validate the problem at the same time and print the results of their validation. I still applied the fundamentals of using stacks in arrays, linked lists in arrays and the STL in arrays.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Formatting, Code Assistance and Explanation of Concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I thought about incorporating all three in a single block of code so I asked Gemini how to do so.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.